

Science of Computer Programming

Protocol-Governed Computing: A Software Development Lifecycle Architecture for Deterministic Declarative Execution and Governed Transformation --Manuscript Draft--

Manuscript Number:	
Article Type:	Research Paper
Keywords:	software architecture; Software Evolution; declarative execution; governance; conformance; provenance; reproducibility; protocol-governed computing
Corresponding Author:	Bhash Ganti Independent Researcher Camas, WA UNITED STATES OF AMERICA
First Author:	Bhash Ganti
Order of Authors:	Bhash Ganti
Abstract:	Software systems are governed across three activities usually treated separately: changing what the system is, constructing an executable representation, and executing it. This separation lets authority migrate between them. Transformation may author behavior, construction may repair incomplete declarations, or a runtime may resolve omissions by inference. Behavior is then determined partly by declarations and partly by the mechanisms that realize them. This paper presents a Software Development Lifecycle (SDLC) architecture for Protocol-Governed Computing (PGC), applying one semantic schema to all three activities. Governed transformation determines next declarations from a named predecessor and stated purpose. Governed construction determines whether candidate declarations may exist and produces an authorized representation. Execution realizes a sealed snapshot without adding behavioral authority. The activities remain distinct in subject and authority, composing through a governed predecessor-to-successor transition. Each transition produces evidence; a refused transition produces no usable governed result. Four contributions follow. The composition is formulated as an SDLC architecture rather than a runtime design. A conformance view is derived from the PGC specification family: closure, reachability, inspection, refusal, evidence, and externally authored profiles. A named reference realization is assessed against obligations its specification and runbook make checkable. Architectural requirements, normative obligations, implementation observations, and evidentiary claims are kept distinct. The assessment is bounded: the realization demonstrates satisfiability and exercised behavior, not universal correctness, business-intent correctness, security, independent-implementation conformance, or maintenance economics. Checks red by design or advisory are reported rather than resolved into a binary maturity claim.

Bhash Ganti · Independent Researcher, Camas, WA, USA · ORCID 0009-0007-3810-6520 · omnibachi.org

To the Editors, Science of Computer Programming

Re: Submission — “Protocol-Governed Computing: A Software Development Lifecycle Architecture for Deterministic Declarative Execution and Governed Transformation”

Dear Editors,

I submit the manuscript above for consideration as a Research Paper. It reaches you through Elsevier’s transfer service from the Journal of Systems and Software (JSSOFTWARE-D-26-02392); as the transfer registered without content, I am submitting it afresh.

What the paper is about. Software behavior is rarely determined in one place. Some is stated in declarations; some is supplied by build tooling, runtime defaults, environment discovery, or fallback behavior. The paper treats this as a question of where behavioral authority resides across the lifecycle, and presents an architecture in which one semantic schema governs all three activities: transformation determines the next declarations from a named predecessor, construction determines whether candidate declarations may exist and emits a sealed snapshot, and execution realizes that snapshot without adding behavioral authority. Each transition produces evidence; a refused transition produces no usable governed result. The architecture is then reduced to a compact set of checkable conformance obligations and assessed against a named, publicly deposited reference realization.

Why Science of Computer Programming. SCP’s scope covers methods for the entire life cycle of software systems, and this paper’s subject is that life cycle taken as one governed object rather than as separate phases. Its contribution sits in the journal’s experimental and descriptive lines of work rather than its formal one.

The distinction matters for how the paper should be read. Its normative apparatus — the specification family the architecture is defined by — is deposited separately and cited, not developed here. What this paper contributes is the architecture, a conformance view derived from it, and an assessment of an executable realization against that view. The realization is a working toolchain: a compiler, a snapshot assembler, a runtime, an inspector, a transformation toolchain, and domain workloads. The composition assessed is deposited with a DOI and identified by a content-derived snapshot identity, and the manuscript carries a reproduction record by which a reader regenerates that identity from the named revision. The empirical portion is therefore open to inspection rather than asserted, and the results reported include the ones that do not support the architecture.

The paper is bounded in the way this readership expects. It separates what the architecture requires, what the specification makes normative, what the implementation was observed to do, and what the evidence establishes, and it names the claims the

evidence does not support. Checks that are red by design or advisory are reported as such rather than resolved into a binary maturity claim.

What is novel. The lifecycle composition itself is a synthesis of prior deposited work, and the paper says so. The novel contributions are: unifying governed transformation, governed construction, and deterministic execution under a single predecessor-to-successor transition while preserving the distinct authority of each; deriving a compact conformance view — closure, reachability, inspection, refusal, evidence, externally authored profiles — from that composition; and, most importantly, a discipline of claim separation that prevents the presence of a governance mechanism from standing in for evidence that the mechanism secures the property it was built for.

I am an independent researcher writing from more than forty years in large-systems integration as a systems architect. The problem addressed here is one I met repeatedly in practice long before it had a name: systems that work while no single artifact determines what they may do. That vantage is what the paper contributes in place of a laboratory.

Affiliation. I hold no institutional affiliation of any kind. My research is published under my own name at omnibachi.org, which is my research website and the only body I am associated with. Correspondence should be directed to me personally.

The manuscript is original, is not under consideration elsewhere, and has not been published previously. A companion study by the same author, cited in §1.3 for disclosure, is under concurrent review elsewhere; it addresses a different question and reuses no text, argument, or evidence from this paper. Prior PGC preprints synthesized here are deposited and cited by DOI. There is no funding and no conflict of interest. Use of a generative AI assistant is disclosed in the manuscript.

Thank you for your consideration.

Sincerely,

Bhash Ganti

Protocol-Governed Computing: A Software Development Lifecycle Architecture for Deterministic Declarative Execution and Governed Transformation

Bhash Ganti

bachipeachy@gmail.com · ORCID [0009-0007-3810-6520](#)

Preprint: <https://doi.org/10.5281/zenodo.22758703>

Abstract

Software systems are governed across three activities usually treated separately: changing what the system is, constructing an executable representation, and executing it. This separation lets authority migrate between them. Transformation may author behavior, construction may repair incomplete declarations, or a runtime may resolve omissions by inference. Behavior is then determined partly by declarations and partly by the mechanisms that realize them.

This paper presents a Software Development Lifecycle (SDLC) architecture for Protocol-Governed Computing (PGC), applying one semantic schema to all three activities. Governed transformation determines next declarations from a named predecessor and stated purpose. Governed construction determines whether candidate declarations may exist and produces an authorized representation. Execution realizes a sealed snapshot without adding behavioral authority. The activities remain distinct in subject and authority, composing through a governed predecessor-to-successor transition. Each transition produces evidence; a refused transition produces no usable governed result.

Four contributions follow. The composition is formulated as an SDLC architecture rather than a runtime design. A conformance view is derived from the PGC specification family: closure, reachability, inspection, refusal, evidence, and externally authored profiles. A named reference realization is assessed against obligations its specification and runbook make checkable. Architectural requirements, normative obligations, implementation observations, and evidentiary claims are kept distinct.

The assessment is bounded: the realization demonstrates satisfiability and exercised behavior, not universal correctness, business-intent correctness, security, independent-implementation conformance, or maintenance economics. Checks red by design or advisory are reported rather than resolved into a binary maturity claim.

Keywords: software architecture; software evolution; declarative execution; governance; conformance; provenance; reproducibility; protocol-governed computing

1. Introduction

1.1 The problem

Software behavior is rarely located in one place. Some behavior is stated in requirements, policies, schemas, workflow descriptions, and configuration. Some is embedded in implementations. Some is supplied by build tools, deployment conventions, runtime defaults, environment discovery, or recovery behavior. The resulting system can work while no single artifact determines all of what it may do.

This distribution of authority is manageable when implementation changes slowly and the people who construct the system can inspect the relevant paths. It becomes harder when implementation is generated, modified, or regenerated at machine speed. A generated implementation can be locally plausible while introducing a route, default, effect, or dependency that no governing declaration admitted. A runtime can make the behavior appear robust by supplying a fallback, but the fallback then becomes part of the system's effective semantics without being governed as such.

The central architectural question is therefore not only how to execute software, or how to change it, but where the authority to determine behavior resides across the whole lifecycle. A system that moves authority into declarations but leaves construction or execution free to infer has not completed the move. A system that makes execution deterministic but allows its declarations to change outside a governed transition has bounded only one part of its lifecycle.

Protocol-Governed Computing (PGC) addresses this problem by carrying governance inside the software as explicit, versioned, machine-consumable declarations. Its specification family defines concepts, relations, invariants, execution semantics, construction obligations, transformation semantics, profiles, and conformance evaluation. Its reference realization supplies a compiler, snapshot assembler, runtime, inspector, transformation toolchain, and domain workloads.

This paper asks:

How can construction, execution, and evolution be specified as one governed lifecycle in a way that separates architectural obligations from implementation observations and makes conformance claims independently checkable?

The answer is not that the three activities are one operation or one component. They have different subjects and different authorities. The answer is that they instantiate one semantic transition schema and are composed through the governed state that each activity produces for the next.

1.2 The lifecycle claim

The PGC conceptual model describes a governed system's life as three activities over four states. Fig. 1 shows the arrangement and the loop that closes it.

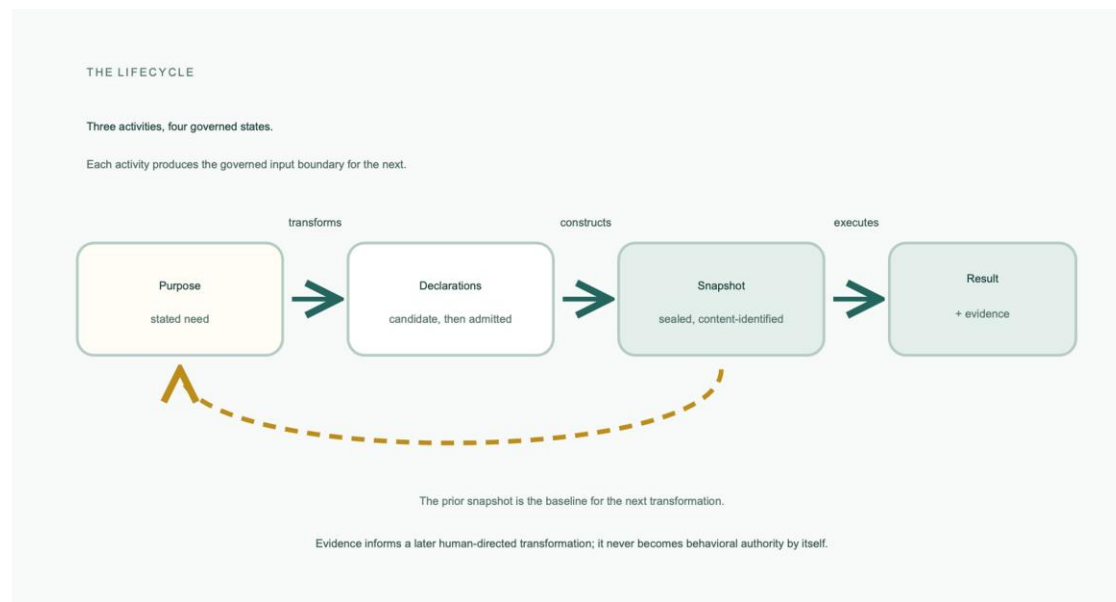


Fig. 1

Fig. 1. Three activities over four governed states. Purpose is transformed into declarations, declarations are constructed into a sealed snapshot, and the snapshot is executed to produce a result and evidence. Each activity produces the governed input boundary for the next, and the prior snapshot is the baseline the next transformation starts from. Evidence informs a later human-directed transformation; it never becomes behavioral authority by itself.

Transformation determines what the declarations are. Construction determines which candidate declarations may exist and produces the authorized representation. Execution realizes what the sealed representation already determines. These are not implementation stages required by the specification. They are semantic activities. A realization may combine or divide them, provided that it does not transfer their authority.

The synthesis made in this paper is a **distinct-but-composed** account:

- Transformation and initial construction are instances of a governed transition from a named predecessor state, or the empty state at genesis, to a successor candidate.
- Construction determines admissibility and produces the representation eligible for sealing; it does not determine whether the candidate fulfills the business purpose.
- Sealing constitutes the snapshot that execution consumes. Compilation and assembly are implementation mechanisms that can realize construction and sealing; they are not themselves mandatory normative stages.
- Execution determines execution-local outcomes under the sealed representation, but it does not originate the workflow structure, routing rules, effects, defaults, or recovery paths.

The lifecycle is consequently more than a list of three components. Each activity produces a governed object or state that supplies the input boundary for the next. The

transition relation is the composition mechanism; the activities remain separate because the authority they hold remains separate.

1.3 Source relationship and bounded novelty

The paper synthesizes three PGC preprints, each publicly deposited and cited by DOI below. The deterministic declarative execution paper develops the Execution Partition: the compiler determines behavior and the runtime realizes it. The closed-loop governed transformation paper develops the Knowledge Partition, staged derivation, the baseline as predecessor, and promotion as the moment of change. The realization paper develops the Profiled Normative Platform, checkable properties of closure, reachability, inspection, and evidence, and the composed functions of transformation, compilation, and execution.

The paper also consulted the Field Manual and five earlier PGS papers covering architecture inversion, the compiler, the conceptual model, the constitutionally constrained architecture, and the runtime conceptual model. Those papers establish historical terminology and prior disclosure. They are not treated as independent empirical studies.

A companion study by the same author, *Protocol-Governed Human-AI Software Engineering: Autonomy Without Authority*, is under concurrent review elsewhere. It evaluates agent-mediated transformation under this architecture and asks a different question: what remains of authorization authority when an agent performs the engineering activity. It is cited here for disclosure and positioning. No text, argument, data, or evidence from it is reused, and the authority/activity partition appears in this paper only as a consequence of the lifecycle architecture, never as a contribution of it.

The lifecycle composition is therefore a synthesis, not a wholly new discovery. The candidate journal additions are the explicit separation of normative obligations from implementation observations, the compact conformance framing, and the bounded assessment against the current specification and reference realization. Those additions are claims only to the extent that the underlying specification revision, profile, workload, and evidence artifacts are identifiable and independently inspectable.

1.4 Contributions

This paper makes four contributions:

1. **A lifecycle architecture.** It connects governed transformation, governed construction, sealed snapshots, and declarative execution through a predecessor-to-successor relation while preserving the distinct authority of each activity.
2. **A conformance framing.** It maps the lifecycle to checkable obligations: governance closure, resolution, snapshot sealing and acceptance, declared reachability, governed inspection, refusal, evidence, and external profile selection.
3. **A bounded realization assessment.** It examines the reference workspace and its named end-to-end runbook, distinguishing demonstrated satisfiability and

exercised behavior from claims that require structural, comparative, or independent evidence not supplied by one realization.

4. **A separation of claim kinds.** It distinguishes what the architecture requires, what the specification makes normative, what the implementation was observed to do, and what the evidence establishes. The four are kept apart throughout, and Appendix E records which of them each claim in this paper rests on. This is what prevents the presence of a mechanism from standing in for evidence that the mechanism secures the property it was built for.

1.5 Reading the paper

Section 2 states the problem as an authority-allocation problem. Section 3 introduces PGC from first principles. Section 4 develops declarative execution. Section 5 develops governed transformation. Section 6 composes the three activities. Section 7 moves from architectural obligations to conformance. Section 8 reports the reference realization and its reproducibility boundary. Section 9 discusses limitations and threats to validity. Section 10 positions the contribution against related approaches. Section 11 concludes. The appendices provide terminology, extracted obligations, a worked lifecycle example, a named-revision reproduction record, and claim provenance.

2. Behavioral Authority in Conventional Software

2.1 A distributed source of behavior

A conventional application can be represented as a set of declarations, implementation code, build artifacts, runtime mechanisms, environmental dependencies, and operational procedures. The system's observable behavior is a function of all of them, even when only some are recognized as specifications:

$$\text{behavior} = \text{declarations} + \text{implementation} + \text{construction choices} + \text{runtime choices} + \text{environment} + \text{state}$$

This expression is not intended as a numerical model. It identifies a governance problem. If a runtime default or a build-time search path changes behavior, then the system has an undeclared behavioral input. If an implementation adds a route absent from the workflow declaration, the implementation has acquired behavioral authority. If a deployment edits a snapshot in place, the deployment has acquired authority over the governed state.

The problem is not that implementation, build, and runtime mechanisms exist. A governed system still needs mechanisms. The problem is that a mechanism can determine a semantic fact without that determination being represented, checked, and evidenced as part of the system's governance.

2.2 Why generation increases the pressure

AI-assisted implementation increases the rate at which code can be produced and changed. It does not by itself create the authority problem; it makes the existing distribution more difficult to inspect. When implementation is treated as the primary behavioral artifact, an evaluator must inspect more generated material, more frequently, while still reasoning about defaults, dynamic lookup, environmental dependencies, and post-deployment state.

A structural response is to move the question earlier: not whether a runtime can stop an inadmissible behavior after it appears, but whether the inadmissible execution surface can be constructed at all. This does not make implementations correct. It makes a different property checkable: whether the implementation is being used inside an execution surface whose behavior and effects were declared and admitted before execution.

2.3 The required distinctions

A lifecycle architecture must preserve at least five distinctions:

1. **Meaning and derivation.** A machine may derive a representation from a human-directed decision, but derivation must not become an independent source of meaning.
2. **Admissibility and adequacy.** Construction may determine whether a declaration may exist; it must not silently decide whether the declaration fulfills an unstated purpose.

3. **Determination and realization.** A runtime may evaluate a declared capability and produce a declared outcome; it must not determine the structure it traverses.
4. **Evidence and authority.** Evidence can establish what was determined or occurred; it does not grant permission for a future action.
5. **Conformance and correctness.** A system can conform to a profile whose declarations are unsuitable for a business purpose. Conformance is a claim against stated obligations, not a proof of intent correctness.

PGC treats these distinctions as semantic and normative boundaries rather than as recommended process discipline.

3. Protocol-Governed Computing

3.1 The conceptual model

PGC defines a governed system as one whose behavior and construction are determined by explicit declarations rather than by the code that realizes them. The declaration is not documentation about a decision made elsewhere. It is the machine-consumable statement that determines what may exist or occur.

The PGC model distinguishes four levels:

Level	Meaning
Governance	The discipline of establishing, enforcing, verifying, evolving, and retiring conditions under which software may act and change.
Governance system	The authorities, rules, mechanisms, and evidence governing one software system.
Governance artifacts	The declarative representations carried by the system.
Behavior	The execution that results from the declarations.

PGC standardizes the arrows among these levels. It specifies how governance is carried as artifacts and how those artifacts determine construction and execution. It does not require one file format, one language, one compiler architecture, or one runtime implementation.

3.2 Governed state and transitions

The semantic model represents a governed state as a complete assignment of values to everything within the applicable governance closure. A proposal is a candidate change presented to that state. A determination evaluates the rules supplied by the closure and yields one of three consequences: admit, constrain, or refuse. The resulting transition carries evidence. Fig. 2 states the schema and the five properties every governed transition must have.

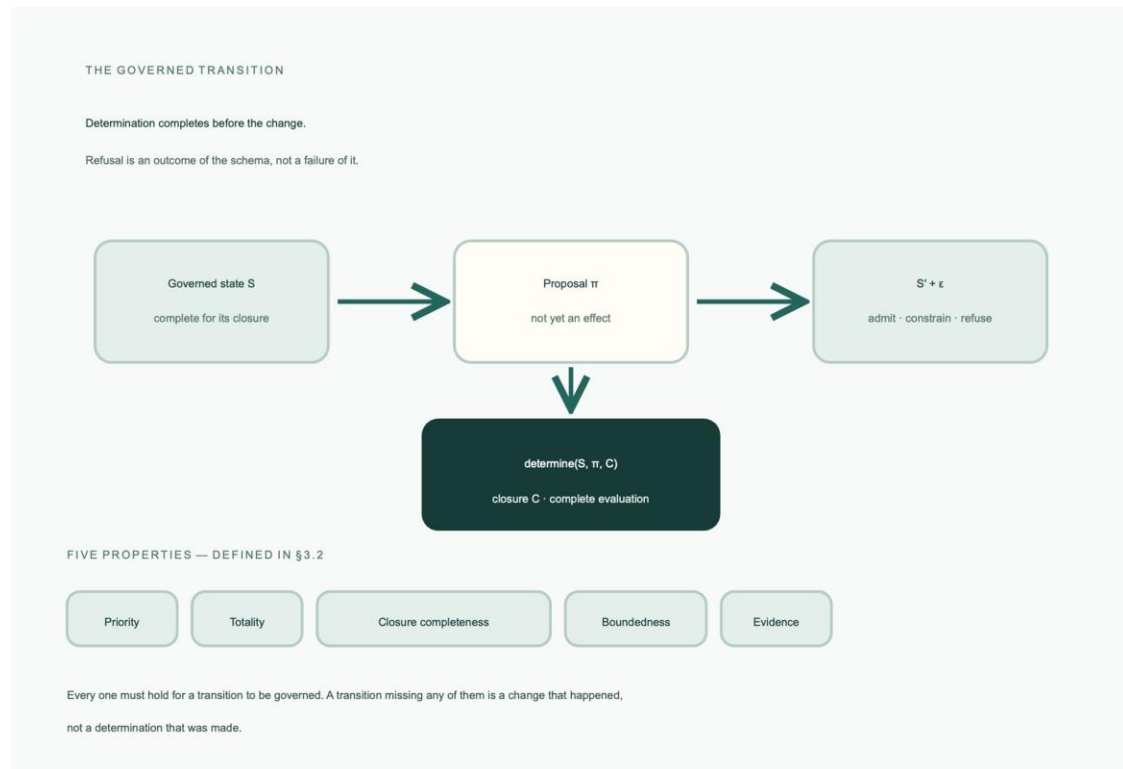


Fig. 2

Fig. 2. The governed transition. A proposal against a governed state is evaluated over the complete applicable closure, yielding a successor state and evidence. The determination admits, constrains, or refuses; refusal is an outcome of the schema rather than a failure of it. The five properties are named here and defined below.

A governed transition has five properties:

- **Priority:** determination completes before the change it governs occurs.
- **Totality:** every proposal receives a determination; no proposal is silently ignored.
- **Closure completeness:** every applicable rule supplied by the closure is evaluated.
- **Boundedness:** the rule set is finite and known before evaluation.
- **Evidence:** the determination and result can be established by a party that did not observe them.

Refusal is therefore an outcome, not an exceptional failure of the governance mechanism. A refused proposal transitions to a state unchanged with respect to that proposal, with evidence of the refusal. A construction that refuses must produce no usable authorized representation. An execution that encounters an unrouted outcome or unresolved reference must not invent a path.

3.3 Governance closure

Governance closure is the complete set of governing elements applicable to a subject, together with their authority, scope, and composition. The standard makes closure

1
2
3
4 distinct from the rule set it supplies. Flattening the distinction would preserve the rule
5 but lose the reason the rule governs.
6

7 The decisive property is asymmetric treatment of uncertainty. An unknown closure is not
8 an empty closure. If an applicable governing element cannot be resolved, authority is
9 undeclared, or the rule set cannot be bounded, the determination is refusal. Treating
10 inability to establish governance as permission is the path by which ungoverned
11 behavior enters a system while remaining superficially indistinguishable from governed
12 behavior.
13

14 15 3.4 Two architectural partitions 16

17 Two named partitions carry the architecture:
18

- 19 • **Execution Partition:** the compiler or construction mechanism determines
20 behavior; the runtime realizes the sealed representation.
21
- 22 • **Knowledge Partition:** only a human-directed author creates meaning; machines
23 may derive, transform, check, and compile, but may not originate meaning.
24

25 The partitions are related but not identical. The Execution Partition concerns where
26 behavioral determination resides. The Knowledge Partition concerns where semantic
27 meaning enters the lifecycle. Together they prevent authority from reappearing below or
28 above the snapshot boundary.
29

30 31 3.5 What PGC is not 32

33 PGC is not a policy engine layered on an otherwise unconstrained application, a
34 workflow orchestrator that plans at runtime, a runtime authorization framework, or a
35 repository convention. A policy engine may evaluate a rule at runtime; PGC requires the
36 governed determination and its consequences to be part of the architecture. A workflow
37 engine may synthesize or retry a path; PGC requires the path and its failure behavior to
38 be declared before traversal. A repository may contain declarations; it is not a platform
39 until a governed composition under a named profile is constituted.
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65

4. Deterministic Declarative Execution

4.1 The executable protocol and snapshot

PGC execution consumes a sealed snapshot. The snapshot is a portable governed artifact that is sealed, complete, self-identifying, self-describing, and verifiable. Its identity is derived from its content. Changing any constituent produces a different snapshot rather than modifying the existing one.

Execution acceptance is itself governed. Before anything runs, the accepting agent must establish:

1. each constituent has the declared integrity;
2. the content derives the identity the snapshot bears;
3. the constituent enumeration is total; and
4. the claimed profile is one under which the accepting context is conformant.

A failure refuses the whole snapshot. There is no partial acceptance, warning-only acceptance, or execution against the parts that happen to verify. This rule makes the snapshot an authority boundary rather than a convenient package of runtime data.

4.2 Traversal without decision

Declarative execution does not mean that no computation occurs. Capabilities may perform computation. It means that execution does not originate behavioral rules. The sealed representation contains the workflow structure, capability bindings, declared inputs, outcome vocabulary, routing, state transitions, and effect surfaces needed for the run.

Traversal reads the declared structure, as Fig. 3 shows.

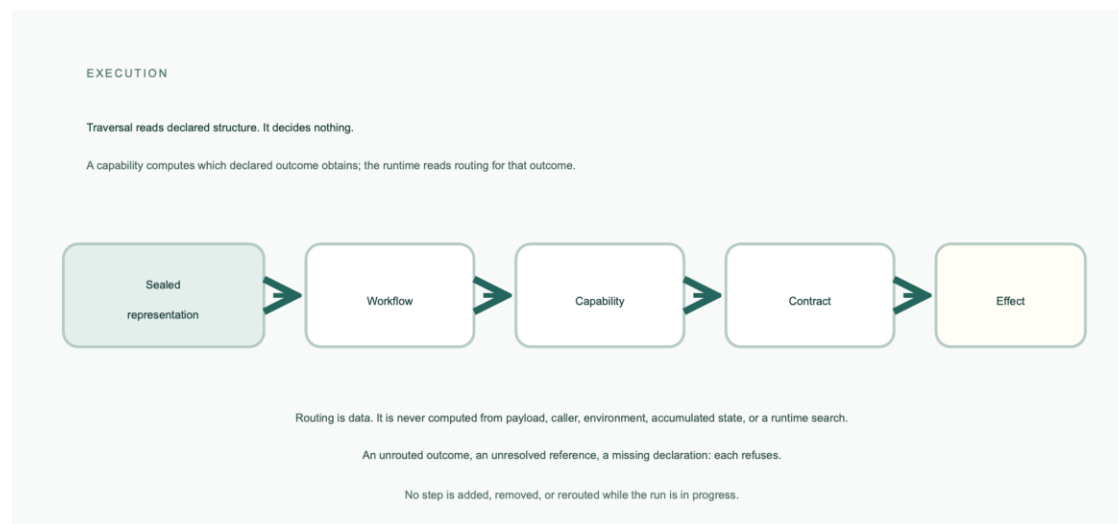


Fig. 3

Fig. 3. Traversal without decision. The runtime reads the sealed representation to reach a workflow, a capability, its governing contract, and a declared effect surface. A

1
2
3
4 capability computes which of its declared outcomes obtains; the runtime reads the
5 declared routing for that outcome and originates none of it.
6

7 Routing is data. A step produces one of the outcomes declared by its governing
8 contract, and that outcome selects the next declared step. Routing must not be
9 computed from a payload, caller identity, environment, accumulated state, or a runtime
10 search. A step cannot be added, removed, or rerouted while the run is in progress.
11

12 The distinction is important. A capability can compute which of its declared outcomes
13 obtains. The runtime can then read that outcome and follow declared routing. What the
14 runtime cannot do is invent an outcome vocabulary, reinterpret a returned value as a
15 route, or use an undeclared failure as permission to choose a fallback.
16
17

18 4.3 Refusal and closed behavior 19

20 The negative path is part of the protocol. If a capability can fail, the failure outcomes and
21 their routing must be declared. An unrouted outcome is a refusal. An unresolved input
22 reference is a refusal. A missing declaration, an invalid snapshot, and a violated
23 obligation are refusals. The runtime does not select a default path, silently halt, retry by
24 convention, or degrade into an unspecified mode.
25
26

27 A capability outcome named `rejected` is not the same thing as a governance refusal.
28 The former is a declared result that traversal may route on. The latter is the
29 determination that a proposal or interaction was not permitted and that nothing
30 proceeded. Evidence must preserve this distinction because the two consequences are
31 opposite.
32
33

34 4.4 State and effects 35

36 Governance defines state; execution maintains it. A write, its target, and its conditions
37 are determined before execution. External effects occur only through declared,
38 governed surfaces. A non-effecting capability cannot acquire an effect path indirectly
39 through the runtime or its environment.
40
41

42 This closes a route by which authority could re-enter through state. What a system may
43 become is part of what it may do. If the runtime owns the choice of state transition, the
44 runtime still holds behavioral authority even if its workflow topology is static.
45
46

47 4.5 Evidence of execution 48

49 Execution evidence must establish the closure and authority that applied, the rules
50 evaluated, each predicate result, the dominant consequence, the resulting state, and
51 the path taken. It must identify the snapshot under which the execution occurred.
52

53 The evidence standard distinguishes deterministic content from observational content.
54 Deterministic content includes the closure, rules, consequence, resulting state, and
55 path; it must be identical for the same state, interaction, and closure. Observational
56 content such as time, duration, location, and environmental measurements may differ,
57 but must not participate in determination.
58
59
60
61
62
63
64
65

This distinction supports replay comparison without pretending that two executions occur in the same physical environment. It also prevents a log that merely records a result from being mistaken for evidence that the result was governed.

4.6 Limits of the execution claim

The Execution Partition does not prove that the declared behavior is desirable, that the capability implementation is correct, that input data is valid, or that the environment is secure. It establishes a boundary around where behavioral determination is allowed to occur. Correctness against business intent remains a separate question. Security claims require their own threat model and structural or comparative evidence.

5. Closed-Loop Governed Transformation

5.1 Transformation starts from a baseline

Transformation is the governed transition from an existing baseline and a stated need to the next baseline. A baseline is not merely context for the change. It is the current governed state: the source of reusable structure, the boundary against which claims about existing behavior are grounded, and the predecessor whose identity must be accounted for. Fig. 4 shows the loop and what it yields when it does not complete.

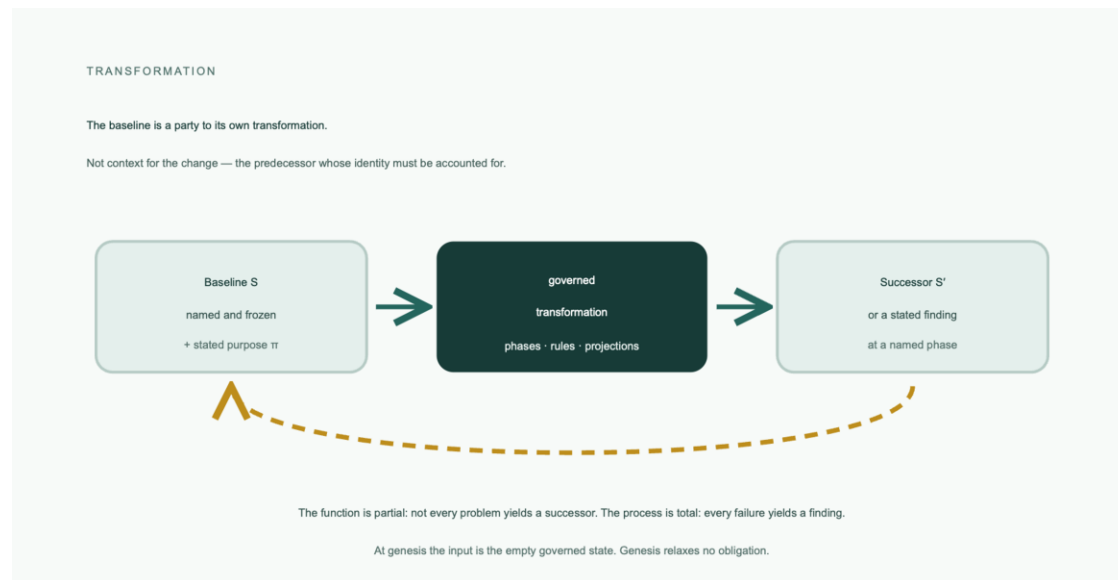


Fig. 4

Fig. 4. Closed-loop governed transformation. A named, frozen predecessor baseline and a stated purpose enter a governed transformation, which yields either a successor baseline or a stated finding at a named phase. The function is partial and the process is total (§6.4).

At genesis, the input is the empty governed state. Genesis is not a relaxation of the obligations. The first transformation must still name an externally authored profile and produce a complete, verifiable first baseline. Every later transformation must use a named frozen baseline and must not claim genesis merely because a new domain or subsystem is convenient to introduce.

5.2 Human meaning and machine derivation

The Knowledge Partition places human-directed judgment at the point where meaning is irreducible. A worker may draft a phase document, derive a projection, check a rule, or compile an artifact. It may not decide what a business need means, fill an unanswered question, or silently replace a human commitment.

The transformation standard expresses this through registers, rules, phases, and gates. Governed content lives in addressed fields rather than in uncheckable prose. A missing value becomes an open question against a named owner. A blocking question makes

1
2
3
4 the phase inadmissible. A human answer is recorded as declared content so that later
5 phases can preserve it, reference it, or explicitly supersede it.
6

7 This is not an argument that prose is useless. Prose can explain, motivate, and
8 interpret. It is not allowed to carry a governed field that the system later treats as settled
9 without an addressable declaration.
10

11 5.3 Phases, rules, and projections 12

13 A transformation is organized as phases ordered by dependency. The ordering is
14 semantic, not necessarily linear. A phase reads what precedes it, emits what it declares,
15 and is judged by a declared rule set. A dossier records the phase documents and
16 determinations, but remains evidence of the transformation rather than a member of the
17 resulting governed system.
18

19 Rules are data. Each rule identifies its register, check kind, and parameters. Check
20 kinds are mechanisms and carry no business policy. Every declared rule must be
21 evaluated, and every rule must be demonstrated capable of refusal. A rule set that has
22 never rejected a malformed register may be silently checking the wrong field, an absent
23 field, or a longer name matched by an imprecise lookup.
24

25 Where a phase is uniquely determined by its prior, it is a projection and must be
26 generated mechanically. A projection cannot launder an inadmissible prior into a clean-
27 looking successor. If its source is inadmissible, it refuses. If it discovers a question, that
28 question returns to the phase that owns it rather than being introduced into the
29 projection as an invented fact.
30

31 5.4 Grounding and preservation 32

33 Transformation must distinguish three kinds of content:
34

- 35 • a business truth supplied by the human;
- 36 • a belief about what the baseline currently provides; and
- 37 • an open question that has not yet been answered.
38

39 A belief about the baseline must be grounded through a declared inspection interface.
40 An inventory is not enough when the transformation needs to compare against a named
41 artifact. The inspection surface must answer questions about specific named artifacts
42 and must not execute or mutate the system it reads.
43

44 Preservation is bidirectional. A later phase must not silently drop what an earlier phase
45 committed, but it must also not fabricate a fact that the earlier phase did not state. A
46 pipeline that checks only for loss can allow an invented fact to pass as successfully as a
47 preserved one.
48

49 5.5 Sufficiency, realization, and proof 50

51 Transformation separates design sufficiency from realization. Sufficiency asks whether
52 the design fixes every fact required to construct an artifact. Realization turns that design
53 into artifacts. A generator that fills a missing design fact is a second design authority
54 and violates the Knowledge Partition.
55

1
2
3
4 A realized artifact must be a function of the design alone. It must not vary because the
5 current baseline happens to contain a different artifact, except where the design
6 explicitly determines a comparison or amendment. An amendment is a whole
7 redeclaration, not an ungoverned delta. The realization schedule must cover every
8 requested artifact and place dependencies before dependents.
9

10
11 Admissibility and sufficiency are not proof of adequacy. The transformation is not
12 complete until the produced system is executed against real state and its declared
13 acceptance criteria are observed. A criterion about data must operate on data. A
14 criterion that only observes a success code does not establish that the intended state
15 was reached.
16

17 5.6 Promotion and supersession

18
19 Normatively there is one act: a governed transition in which a snapshot is replaced by a
20 successor rather than modified in place. *Promotion* is the operational name for the
21 moment that transition takes effect, and it is used in this paper only in that descriptive
22 sense. It confers nothing. A mechanism that promotes has executed a determination
23 already made; if promotion could admit what construction refused, or bind a successor
24 that no transformation produced, it would be a second construction authority under an
25 operational name.
26
27

28
29 Supersession is a declared relation between exact identities. It is not a redirect rule. A
30 reference to the predecessor continues to name the predecessor. Within the governed
31 composition, the predecessor becomes unreachable by execution and ordinary
32 reference but remains retained and inspectable. The supersession declaration itself
33 names the predecessor because that naming establishes retirement; unrelated
34 references to the predecessor are refused.
35
36

37 This preserves historical evidence without allowing an old identity to remain load-
38 bearing. It also prevents a deployment mechanism from silently changing what an
39 existing reference means.
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65

6. The Unified Lifecycle

6.1 One semantic schema, three activities

The PGC semantic model applies one governed transition schema to three subjects:

Activity	Governed state S	Proposal π	Result S'	Determines
Transformation	Current baseline	Stated need and required human answers	Next baseline	what the declarations are
Construction	Authorized declarations	Candidate declaration	Authorized representation	which declarations are authorized to execute
Execution	Governed state under a sealed snapshot	Interaction	State after the interaction	what occurs under those declarations

The rightmost column is the reason the activities are distinct: they determine different things, and no one of them may determine another's subject.

They are composed because the result of each activity is the governed input boundary for the next. Transformation supplies declarations to construction. Construction supplies an authorized representation to sealing. The sealed snapshot supplies the sole source of governed behavior to execution. Execution produces result and evidence, which can become inputs to a later human-directed transformation but never become behavioral authority by themselves.

6.2 One transition pattern, two authorities

The synthesis claim is strongest at the transition boundary. Initial construction and later evolution are not unrelated operations. Both take a governed state and a candidate change, evaluate a governance closure, produce a successor or refusal, and retain evidence. The difference is the kind of proposal and the baseline available:

- construction receives a candidate declaration for authorization;
- transformation receives a stated need and determines the declarations that should become candidates;
- genesis begins from the empty state, while later transformation begins from a named predecessor baseline.

The activities must not be collapsed. Transformation owns the question of what the declarations should be. Construction owns whether candidate declarations may exist. The composition is therefore **one governed transition pattern with different subjects and authority partitions**, not a single undifferentiated operation.

6.3 Composition, and what in it is not normative

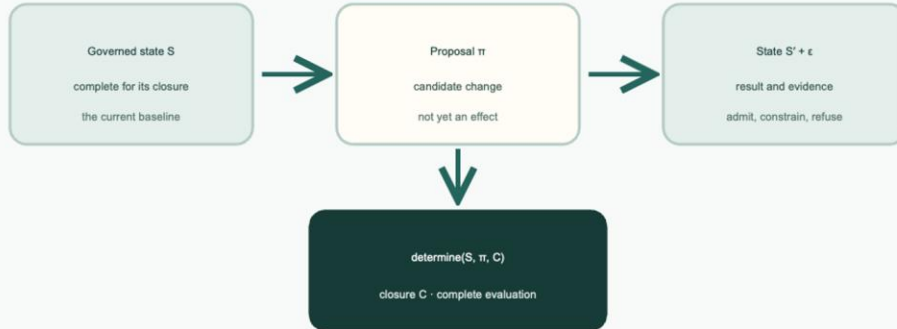
Fig. 5 draws the composition whole: the transition schema, the three activities that instance it, and the authority that keeps them apart.

THE MODEL

One pattern, three activities, separate authorities.

The semantics is shared; the authority each activity holds is not. They compose, they do not collapse.

THE GOVERNED TRANSITION



THE PGC LIFECYCLE

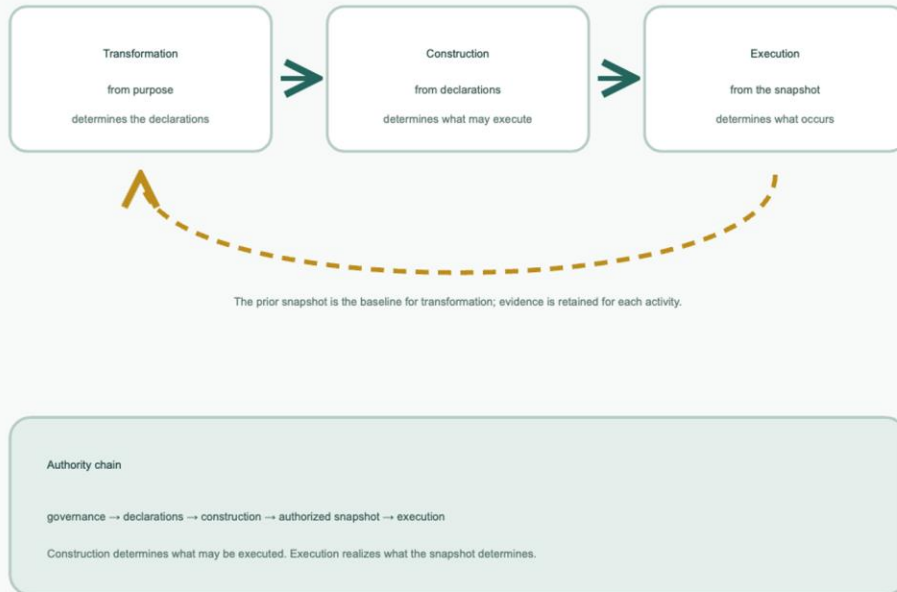


Fig. 5

Fig. 5. One pattern, three activities, separate authorities. The paper's central figure. Above: the governed transition schema, $\text{determine}(S, \pi, C)$, yielding a successor state and evidence. Below: transformation, construction, and execution as three instances of that one schema over different subjects — and the column that keeps them apart, which is what each *determines*. The dashed return records that the prior snapshot is the baseline for the next transformation. The authority chain states the composition's direction: construction determines what may be executed, and execution realizes what the snapshot determines. The semantics is shared; the authority is not.

A concrete implementation may realize T, C, sealing, and E with different packages or with one process. The architecture requires the semantic boundaries to remain observable. In particular, execution cannot be used to decide admissibility, construction cannot repair missing meaning, and transformation cannot become a mechanism for bypassing an external profile.

Compilation is not a normative stage, and this matters for how the rest of the paper should be read. The realization reported in §8 has a compiler and a snapshot assembler because those are useful mechanisms for discharging the construction and sealing obligations. They are not what the specification requires. A conforming realization could fuse them, split them differently, or discharge the same obligations by another mechanism entirely. Every implementation detail in §8 — the compiler stages, the projection formats, the directory layout, the address scheme — is reported to make the subject inspectable, never to define the architecture. Reading the current pipeline as normative would convert this paper into a description of one implementation and would contradict the specification family it assesses.

6.4 Partial transformation and total process

The transformation function is partial. Some stated problems correctly yield no successor because a closure cannot be established, a rule refuses, a design is insufficient, realization cannot be completed, or execution fails the declared acceptance criteria. A partial function is not a defect; it is how governed refusal remains possible.

The process is total in a different sense. Every proposal must end in a stated determination at a named phase. A failure must not become an undefined state, an untracked partial output, or a silent omission. A refused transformation produces a finding and no successor baseline. This distinction avoids the false choice between pretending every problem can be solved and treating a correct refusal as an exceptional failure.

6.5 Invariants of the composition

The composed lifecycle must preserve these invariants:

1. **Declaration origin:** every governed behavior traces to an admitted declaration.
2. **Non-ambient authority:** no authority derives from position, containment, load order, or caller proximity.
3. **Activity separation:** transformation, construction, and execution do not assume one another's authority.

4. **Determination before effect:** no governed effect occurs before the determination permitting it.
5. **No discovery:** candidates, references, routes, effects, and dependencies enter only by declaration and admission.
6. **Real sealing:** a snapshot is immutable and content-identified.
7. **Whole refusal:** refusal leaves no usable partial governed result.
8. **Evidence:** every determination, including refusal, is evidenced and independently checkable.
9. **Successor change:** declarations and snapshots change by governed transformation and replacement, not mutation in place.

These invariants are semantic obligations. The reference implementation's compiler stages, directory layout, hashes, JSON projections, and shell scripts are techniques that may preserve them; they are not the invariants themselves.

7. From Architecture to Conformance

7.1 Why conformance is a separate question

An architecture says what must remain true. A conformance claim says that a named subject satisfies those obligations against a named profile and specification revision. The two claims must not be merged. A mechanism can appear to implement an obligation without evidence that the property holds. A passing workload can show an execution result without establishing that no unobserved path exists.

The PGC Conformance Model requires every claim to name:

- the subject;
- the profile;
- the PGC specification revision; and
- the claimant.

There is no unqualified “PGC conformant” claim. A runtime claim, snapshot claim, construction claim, transformation claim, profile claim, and system-instance claim are different claims with different discharge requirements.

7.2 The Profiled Normative Platform

A platform is a governed composition under a named profile. It is not a repository, package, deployment, or installation. The profile selects the facilities of the standards family, narrows choices the family leaves open, supplies parameters, and may add requirements within an explicitly permitted extension point.

The externality of the profile is essential. A snapshot claims a profile, but does not author the profile that determines whether the claim holds. This prevents genesis from becoming self-certification and prevents a later transformation from weakening the profile at the same time it violates it.

Externality has three degrees, and they are worth separating because a realization can satisfy the first while a reader assumes the third. A profile is external **by authorship** when someone other than the claimant wrote it; **by storage** when it is not distributed by, or versioned with, the system claiming it; and **by identity** when the snapshot’s identity covers the profile’s content, so that changing what the profile requires changes what any snapshot claiming it is. Only the third makes the claim tamper-evident. A snapshot whose identity covers the profile’s *name* but not its *content* can be assembled today and assembled again after the profile has been weakened, and carry the same identity both times — the same defect the standard’s own identity clause exists to prevent, occurring one level up, on the document that determines conformance rather than on the artifacts it constrains.

The profile in force for the composition assessed in §8 is GOVERNANCE_SURFACE_PROFILE_V0, which supersedes REFERENCE_PLATFORM_PROFILE_V1. The two are not interchangeable, and §9.3 reports which degrees of externality this realization currently establishes.

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33

7.3 Discharge classes

An obligation is not checkable until it is known what kind of evidence would discharge it. The conformance model distinguishes four kinds:

- **Observational:** exercise the subject and compare what occurred with the obligation.
- **Structural:** inspect a representation or call graph to establish that a forbidden path does not exist.
- **Comparative:** substitute an independent runtime, environment, boundary, or construction context and compare governed consequences.
- **Derivational:** re-derive identity, path, projection, or determination from supplied evidence and representation.

The distinction prevents a common overclaim. A successful run is observational evidence of what happened. It is not structural evidence that an undeclared path cannot happen. A single runtime is not comparative evidence of runtime interchangeability. An internally consistent trace is not derivational evidence that the trace describes the claimed snapshot unless the snapshot identity and relevant closure are carried.

The classes are not interchangeable and cannot substitute for one another. An obligation discharged in the wrong class has not been discharged; it has been answered with evidence about something else.

34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65

7.4 A compact obligation set

The lifecycle argument of §§4–6 maps onto the following obligations. Each is paired with the question an evaluator asks and the class of evidence that would answer it.

Obligation area	Question for an evaluator	Discharge
Closure	Can the complete applicable governance, authority, and rule composition be established?	Derivational, with observation
Resolution	Do all references resolve before the activity that depends on them?	Structural
Construction	Does construction refuse rather than repair, and produce no usable output on refusal?	Negative observational
Sealing	Is identity derived from complete content, and is the snapshot immutable?	Derivational, with structural
Acceptance	Does acceptance establish the four conditions stated in §4.1, and does failure of	Negative observational, with derivational

Obligation area	Question for an evaluator	Discharge
	any one refuse the whole snapshot?	
Reachability	Is execution limited to declared and admitted structure?	Structural, with refusal fixtures
Inspection	Can a read-only boundary answer about named artifacts without executing?	Structural, with observation
Refusal	Do missing, unrouted, undeclared, or unauthorized cases refuse with grounds and no residue?	Negative observational
Evidence	Can a party that did not observe the activity establish what was determined and what occurred?	Derivational
Transformation	Is the next baseline grounded in a named predecessor and produced without invented meaning?	Structural, with observation

Two things this table is not. It is not a substitute for the standards family, which states each obligation normatively and in full. And it is not a checklist whose completion constitutes conformance: an obligation can be listed, demonstrated in the right class, and still inadequately discharged if the demonstration was incapable of failing (§7.5). Appendix B gives the same obligations at the finer grain a claim would need, splitting those that carry more than one requirement.

7.5 Negative demonstrations

The PGC conformance test specification makes negative demonstrations mandatory where obligations refuse or prohibit something. A refusal demonstration must show the refusal, its grounds, its cause, and that nothing partly proceeded. An absence demonstration must state the search space and establish that the search was total over that space. Every demonstration must be capable of failing, and every fixture must be identified.

This requirement changes how the reference realization should be reported. A green execution workload demonstrates that a valid path can execute. It does not demonstrate that defaults, dynamic discovery, effect bypasses, or undeclared routes are absent. Those claims require negative, structural, or comparative evidence.

8. Reference Realization and Named-Revision Reproduction

8.1 Scope and identity of the realization

Section 7.1 requires every conformance claim to name its subject. This section discharges that requirement before making any claim.

The reference realization is the public protocol-governed-computing organization and its component repositories. The subject of this paper's assessment is:

- PGC standards revision: v0;
- public composition identity: v4;
- frozen composition ordinal: 16;
- install distribution: 4.0.0;
- profile in force: GOVERNANCE_SURFACE_PROFILE_V0;
- snapshot identity:
d92b447fd39eaf926bb2f4330f9efbb19d744ecfbc3fe9a214ae52945918905d;
- composition DOI: [10.5281/zenodo.22714911](https://doi.org/10.5281/zenodo.22714911).

The composition is deposited and citable. Its manifest names the nine component archives as parts, each carrying its own version DOI, and records the commit each was built from — so the subject is identified by a single citable artifact rather than by a list of repository states a reader would have to reassemble.

The ordinal is a property of the composition, not of the workspace. The component repositories and the frozen v4 composition declare ordinal 16; the workspace counter advances to 17 when the next development cycle is cut. The ordinal is carried in a compiled constituent and constituents enter the snapshot identity, so a rebuild after the counter advances seals a different identity over byte-identical governed content: the seven domain graph addresses and the 595 constituents are unchanged, and only the ordinal differs. The assessed subject is therefore the v4 composition frozen at ordinal 16, not the later development workspace at ordinal 17. A reproduction must build at the composition's ordinal rather than whatever the workspace currently declares.

The realization contains governance declarations, a protocol compiler, snapshot assembler, runtime, transport boundary, snapshot inspector, transformation toolchain, conformance workloads, and business domains. The composition seals seven domains: the governance surface, the Collatz conformance workload, transformation, snapshot inspector, AI governance, book library management, and blockchain.

Two deposits exist and are not interchangeable. The preceding deposit carries public identity v3 and profile REFERENCE_PLATFORM_PROFILE_V1; the composition assessed here carries v4 and GOVERNANCE_SURFACE_PROFILE_V0, which supersedes it. Both were sealed at assembler ordinal 16, so the ordinal does not distinguish them — public identity and profile do. Every claim below concerns the v4 deposit; nothing in this paper is a claim about v3, and a reproduction must not substitute one for the other.

These facts identify the subject. They do not by themselves establish conformance.

8.2 How the implementation discharges the obligations

Construction and sealing are discharged by a compiler that projects admitted declarations into materialized artifacts and by an assembler that composes compiled roots under a named profile into a manifest-pinned snapshot. Execution is discharged by a runtime that accepts and verifies that snapshot, traverses its declared structure, invokes declared capability implementations, and emits traces. Inspection is discharged by a read-only query surface over the sealed representation. Transformation is discharged by a separate toolchain carrying phase documents, rule sets, projections, and execution testbeds.

Per §6.3, none of this is the architecture. The obligation is what the specification states; the mechanism is one way to discharge it, and a different realization discharging the same obligations differently would conform equally. The mapping is given so that the subject of §8.3–§8.6 is identifiable, not to define what a realization must contain.

8.3 The named-revision build

The documented procedure is a single command that performs a clean rebuild, every check, and the execution block:

```
cd <workspace-root>
bash .github/process/regression.sh --all
```

The word *clean* is load-bearing and is the reason the procedure is given as one command rather than as a compile sequence. Before building, the script deletes every generated snapshot — each domain’s as well as the assembled one — removes the runtime `data/` and `traces/` roots, and sweeps `.DS_Store` workspace-wide. Each step closes a way for a stale result to be reported as a fresh one. A retired artifact left in a domain’s compiled output would be carried into the next composition and would surface as an undeclared output, which reads as a compiler defect rather than as stale state. A stray `.DS_Store` under the sealed deposit would make acceptance refuse it as carrying undeclared content: correct behavior, indistinguishable to an outside reader from a corrupt archive.

The build the script performs expands to compilation of the platform configuration, which yields the governance-surface domain, then compilation of the six remaining domains, then assembly under the profile in force, with `PGC_SNAPSHOT_PROFILE=GOVERNANCE_SURFACE_PROFILE_V0` in the environment. Appendix D records the expansion, the subject identity, and the outputs to capture. One run claims exactly one profile, because a snapshot claims exactly one profile identity.

A clean run is not an entirely green run, and the distinction is deliberate. The runbook designates certain checks as expected-red or advisory and states that a run reporting no red at all means something has stopped reporting. The run recorded here contains two reported red categories, for different reasons.

Admission-contract fidelity reports 31 findings over 402 authored artifacts compared. Each names a gate whose declared payload and the workflow that binds it disagree — a gate admitting a field the workflow cannot resolve, or requiring one that no step

1
2
3
4 consumes. These are reported rather than repaired because the check's purpose is to
5 keep the disagreement visible at the boundary where it is decidable; a run that reported
6 none would mean the comparison had stopped being made.
7

8 *Snapshot validation* returns `valid: true` over ten checks with no non-advisory violation,
9 and two advisory checks `red: republished_copies_agree` at 15 violations, and
10 `bound_paths_declared_as_stores` at 1. The first reports that every capability side-effect
11 artifact published twice, once as the governance surface's authoring copy and once as
12 a consuming domain's execution binding, diverges on content, layer, references, and
13 version; its count scales with the number of domains composed. The second reports a
14 binding to a path that no store declares. Advisory failures exit zero; `--strict` makes
15 them fail.
16
17

18 The advisory classification is itself worth stating plainly, because it is the kind of thing a
19 conformance claim can hide behind. These are observations the composition does not
20 satisfy. They are advisory because the runbook has decided they do not block, not
21 because they are absent. Reporting `valid: true` without them would be accurate and
22 misleading at the same time. No claim of full conformance is made anywhere in this
23 paper.
24
25

26 8.4 Existing end-to-end workloads

27 The runbook's execution block exercises the following paths:
28

- 29 • the Collatz workload through `WF_COLLATZ_CONJECTURE_V0`;
- 30 • AI governance agent action through `WF_GOVERN_AGENT_ACTION_V0`;
- 31 • AI licensing provisioning through `WF_PROVISION_AI_LICENSING_V0`;
- 32 • book library catalog execution in two separate data roots;
- 33 • blockchain identity and wallet execution in one domain data root.

34 The runbook states expected successful results for the positive paths, including Collatz
35 termination, agent-action success, licensing provisioning success, and the domain
36 validation suites. It also runs transformation phase testbeds, projection and construction
37 acceptance checks, inspector checks, implementation closure checks, and environment
38 checks.
39

40 The correct interpretation is bounded: these runs show that the named implementation
41 can build and exercise the supplied composition along the supplied paths, and that
42 selected properties are observable in those runs. They do not show universal
43 correctness, business-intent correctness, security, independent-runtime equivalence, or
44 absence of all undeclared paths.
45

46 8.5 What the reference realization demonstrates

47 Subject to successful execution of the named revision and preservation of its artifacts,
48 the reference realization can support observations about:
49

- 50 • compilation and assembly of a named composition under an external profile;
- 51 • refusal when required build declarations or outputs are missing;

- content-derived snapshot identity and reproducibility where the rebuild comparison passes;
- traversal of declared workflows and production of execution evidence;
- read-only inspection over the sealed representation;
- transformation testbed behavior, including admissible and inadmissible phase documents;
- domain execution over the supplied positive workloads;
- discovery of defects that were not found by architectural reading alone.

The realization also supplies negative evidence about its own limits, and one episode is worth reporting because it illustrates §7.3's distinction between discharge classes rather than merely recording a bug.

An earlier revision of the runtime carried its own acceptance check: it recomputed the composite identity over the *recorded* per-domain hashes in the manifest. That check passed, and the snapshot booted and reported healthy. It was nonetheless the wrong determination twice over. Recomputing over recorded hashes detects a tampered manifest but not a tampered constituent, so a snapshot with an edited projection was accepted; and returning the correct identity is not the same as refusing a manifest that claims a false one. Maintaining a second, weaker implementation of one determination creates two authorities that can disagree.

The current realization has removed the second implementation. Acceptance is one determination reached by one implementation, which recomputes every constituent from its bytes and evaluates all four clauses — integrity, totality, identity derived from the recomputed constituent hashes, and profile. The runtime refuses the whole snapshot on any failure rather than carrying a weaker copy of the test. The current warm-boot testbed reports 6/6 passed on a built snapshot.

Two things follow for the paper's evidence discipline. First, this is a defect that realization surfaced and architectural reading did not: the architecture said acceptance must establish identity from content, and a mechanism existed that appeared to do so while establishing something weaker. Second, a passing acceptance test is observational evidence about the case exercised; it is structural evidence only when the recomputation demonstrably covers the whole constituent set. The distinction is what separated the earlier check from the current one.

Deliberate red findings elsewhere in the runbook serve the same purpose: they show that the procedure distinguishes satisfiability from complete correctness, and that a wholly green run would be a reporting failure rather than a result.

8.6 Current evidence boundary

The current evidence is single-realization evidence. It is not independent-implementation evidence. The same codebase supplies compiler, assembler, runtime, inspector, and much of the testbed, so a passing internal test can be affected by shared assumptions. The current runbook also does not, by itself, provide a second independently implemented runtime, a second protocol boundary, or a second environment sufficient to discharge comparative obligations.

1
2
3
4 Similarly, observed deterministic traces establish determinism for the tested state,
5 interaction, and closure. They do not establish that all possible environmental
6 dependencies are absent. Structural absence claims require total analysis over the
7 relevant search space. Security claims require a threat model, structural reachability
8 analysis, and evidence that the relevant attack classes are represented by the profile
9 and demonstrations.
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65

9. Assessment and Limitations

9.1 What the architecture supports

The architecture supports a precise allocation of authority:

- transformation determines what declarations should constitute the next baseline;
- construction determines whether candidates may exist and refuses rather than repairing;
- sealing makes the authorized representation immutable and content-identified;
- execution consumes the accepted snapshot and realizes only its declared structure;
- evidence makes each determination externally checkable;
- a profile supplies constraints from outside the system claiming it.

The composition is stronger than a collection of process rules because each boundary has a distinct failure condition. A missing answer in transformation is not repaired by construction. An inadmissible declaration is not carried into a snapshot. An invalid snapshot is not partially executed. An unrouted outcome is not sent through a runtime default. A missing evidence record does not become acceptable because the result looked plausible.

9.2 What the architecture does not establish

The architecture does not establish that human-directed meaning is correct, that a profile captures all desirable business constraints, that a capability implementation is functionally correct, or that the environment is secure. It does not make a single realization independently conformant merely because the realization is internally consistent. It does not make conformance a measure of maturity or quality.

Nor does the architecture establish anything about the pipeline that realizes it. The point is developed in §6.3: compilation and assembly are mechanisms for discharging construction and sealing, not normative stages, and nothing in §8 should be read as defining what a conforming realization must contain.

9.3 Threats to validity

Source overlap. The lifecycle composition draws directly on the three PGC companion preprints. The paper makes a bounded synthesis claim and identifies prior disclosure rather than presenting the vocabulary or three-function account as wholly new.

Single implementation. The reference assessment covers one implementation family. It cannot establish independent-implementation conformance or interoperability with a second runtime.

Revision drift. The implementation repository, profile, standards revision, and sealed release do not all share one identity. Results must name the implementation revision and the standards/profile revision separately.

Profile externality is partial in this realization. Against the three degrees distinguished in §7.2, the assessed realization establishes externality by authorship and not by storage or by identity. The profile documents are read from a directory in the same workspace that produces the snapshots claiming them, and the snapshot identity covers the claimed profile's identity string rather than the profile's content. Two consequences follow, and both bear on §7.2's argument rather than on the architecture's requirement. First, a change to what the profile requires does not change the identity of a snapshot that claims it, so the conformance claim is not tamper-evident in the way the artifacts under it are. Second, the anti-self-certification property is upheld by how the profile was authored, which is a fact about the process rather than a property a reader can check from the snapshot. Both are addressable — the profile can be distributed separately and its content hash covered by the identity — and neither is a defect in the architecture, but a conformance claim must not present authorship externality as though it were the checkable kind.

Incomplete negative coverage. Positive workloads cannot establish structural absence. The runbook includes negative and structural checks, but their coverage and red findings must be reported rather than summarized as a maturity score.

Unsatisfied obligations classified as advisory. Two snapshot-validation checks fail on the assessed composition and are classified advisory, so the run exits zero (§8.3). A conformance model that permits a subject to carry unsatisfied checks below a blocking threshold has moved a governance decision into the classification of the check. This paper reports the two findings and their counts rather than the exit status; a stronger treatment would require the profile to state which checks may be advisory and why, which the current profile does not.

Defect history and single-implementation acceptance. Snapshot acceptance was, in an earlier revision, discharged by a second and weaker implementation inside the runtime (§8.5). The current realization reaches one determination through one implementation, and the built-tree warm-boot testbed reports 6/6 passed. That is the appropriate correction to the earlier defect, but it also means the acceptance obligation is now discharged by a single code path shared with the assembler: a defect in that path would not be caught by disagreement between two implementations, because there is no longer a second one. Comparative evidence from an independently implemented runtime remains the discharge this realization cannot supply.

Business-intent validity. The realization can show that a declared criterion was exercised. It cannot independently establish that the criterion was the right expression of the business purpose.

Environment dependence. Reproduction commands constrain the environment, but a single environment does not discharge comparative claims about environment independence. The paper reports the environment as part of the subject and does not generalize beyond it.

10. Related Work

10.1 Model-driven engineering and executable specifications

Model-driven engineering and executable specifications move behavioral content away from low-level implementation (Brambilla et al., 2017), and the field's own assessment of its open problems identifies model/code consistency and tool trust among them (Bucchiarone et al., 2020). PGC shares the movement but makes governance and admissibility part of the executable boundary. Its distinctive claim is not that declarations can generate code; it is that construction and execution must refuse paths not determined by admitted declarations, and that change to those declarations is itself governed. Where MDE typically treats the generator as a productivity mechanism, PGC treats the equivalent step as an authority boundary: a generator that supplies a fact the design omitted has become a second design authority.

10.2 Policy-as-code and runtime policy engines

Policy-as-code makes policy machine-readable and testable, and general-purpose engines such as the Open Policy Agent (2016–) decouple policy from the services that consult it. Admission control in cluster orchestration applies the same idea at a deployment boundary. A policy engine may nonetheless be consulted at runtime by an otherwise independently constructed application, and an unconsulted path is unconstrained. PGC places the boundary earlier: the execution surface is admitted before runtime and the runtime does not supply missing policy semantics. The comparison is not that PGC replaces every policy system; it is that PGC locates behavioral authority in the governed construction of the execution surface rather than in a consultation the application must remember to perform.

The nearest architectural neighbour is capability-based security, which likewise denies ambient authority and requires that the right to act be conveyed rather than assumed (Dennis and Van Horn, 1966; Miller, 2006). PGC's non-ambient-authority invariant is the same commitment applied to a different subject: not the right of a running program to reach an object, but the right of a declaration to determine behavior at all.

10.3 Infrastructure-as-code and declarative deployment

Infrastructure-as-code treats infrastructure declarations as the source for repeatable provisioning. Empirical work on IaC scripts finds that declarative form does not by itself prevent defects or embedded security smells (Rahman et al., 2019, 2020), which is the relevant caution for any architecture that relocates behavior into declarations: the relocation makes behavior inspectable, not correct. PGC shares the emphasis on declarative source, identity, and reproducibility, but applies the governance relation to software construction, execution, evidence, and evolution. The snapshot is not a deployment cache, and replacement is not an in-place mutation. A deployment can instantiate a platform, but it does not become the authority to amend the platform's governed representation.

10.4 Reproducible builds and provenance

Reproducible-build work provides a practical basis for comparing outputs from the same source and has been developed at distribution scale (Lamb and Zacchiroli, 2022). Supply-chain provenance frameworks record and attest the steps by which an artifact was produced (Torres-Arias et al., 2019; Newman et al., 2022; OpenSSF, 2023). PGC uses the same insights while making reproducibility one obligation within governed construction and snapshot identity.

The difference is what the record is allowed to license. An attestation establishes that a named party performed a named step on named inputs; it does not establish that the resulting behavior was admissible. The PGC evidence standard makes this explicit: evidence establishes what was determined or occurred and does not grant permission for a future action. An artifact can carry complete, verifiable provenance and remain inadmissible under the profile it claims. Sealing addresses the complementary question — not who produced the artifact, but whether what it authorizes was determined before execution.

10.5 Formal methods and conformance testing

Formal methods establish properties through mathematical models and proofs (Jackson, 2006); conformance testing establishes obligations through demonstrations. PGC's conformance model combines observational, structural, comparative, and derivational discharge classes and does not claim proof for any of them. Its emphasis on negative demonstrations reflects a practical concern that neither tradition removes: a system that works on valid inputs has not thereby shown that forbidden paths are absent or refused, and a demonstration incapable of failing establishes nothing about the obligation it is offered against.

Sealed-module execution models pursue a related containment goal by fixing an interface and denying the module ambient access to its host (Haas et al., 2017). PGC's snapshot is a boundary of a different kind: it fixes not the module's interface but the whole admissible execution topology, and its acceptance is a governance determination rather than a load-time validation.

10.6 Conventional application architecture

Conventional application architectures typically distribute authority across code, configuration, deployment, and runtime policy. PGC's inversion is to make governance primary and execution derivative: a runtime may be simple because the admissible execution topology is already present in the sealed representation. This shifts complexity toward declarations, construction, profile design, evidence, and transformation. It does not eliminate complexity; it places the complexity where it can be inspected and refused before execution.

11. Conclusion

Protocol-Governed Computing can be understood as a lifecycle architecture whose central object is not a runtime but a governed transition. Transformation determines the next declarations from a named predecessor and a stated purpose. Construction determines whether candidate declarations may exist and produces an authorized representation without repairing omissions or inventing meaning. Sealing constitutes an immutable, content-identified snapshot. Execution accepts that snapshot and realizes its declared structure without adding behavioral authority. Evidence records the determination and occurrence so that a party outside the producer can check them.

The three activities are distinct, but they are not independent. They instantiate one governed transition schema over different subjects and compose through predecessor and successor states. That is the contribution of the lifecycle view: construction, execution, and evolution are not separate sources of behavioral authority, and they are not collapsed into one implementation function. They are composed activities whose boundaries prevent authority from moving silently between them.

The reference realization provides an inspectable demonstration of the architecture's satisfiability and exercised behavior at a named composition identity. It also provides the limits of that demonstration: checks that are red by design, one implementation family, an acceptance determination now discharged by a single shared code path, and no basis for universal correctness or independent-runtime conformance. Those limits are not peripheral. They are part of the paper's claim discipline. A governed architecture is strengthened when its evidence states what it establishes and what it leaves open.

The resulting proposition is therefore bounded but testable:

Behavior may be determined by declarations, constructed into an authorized representation, sealed into a governed snapshot, realized without runtime amendment, and changed only through an evidenced transition from a named predecessor.

Whether a particular system achieves that proposition is not decided by its vocabulary or its intentions. It is decided by the obligations it claims, the profile and revision against which it claims them, and evidence that can establish the claim without requiring trust in the mechanism that produced it.

Appendix A. Key Terms

Admissibility: The construction determination of whether a candidate declaration may exist in the governed system. It is distinct from adequacy.

Baseline: The governed state that represents what the system currently is and supplies the predecessor for transformation.

Construction: The governed activity that determines admissibility and produces an authorized representation.

Determination: The result of evaluating the complete applicable governance closure for a proposal.

Evidence: The semantic record by which a party that did not observe a determination or execution can establish what was evaluated and what resulted.

Execution: The governed activity that realizes behavior under an accepted sealed snapshot.

Governance closure: The complete set of applicable governing elements, their authority, scope, and composition.

Profile: An externally authored selection and constraint over the facilities of the PGC standards family.

Refusal: A determination that a proposal does not proceed. Refusal is an outcome, not an implementation error.

Sealing: The act that constitutes an immutable snapshot and derives its identity from its content.

Snapshot: The sealed, complete, self-identifying, self-describing, verifiable representation consumed by execution.

Supersession: A declared relation between exact identities in which a successor stands down a predecessor without redirecting references.

Transformation: The governed activity that determines the next declarations and baseline from a named predecessor and a stated purpose.

Appendix B. Extracted Conformance Obligations

The finer grain of §7.4, for obligations that carry more than one requirement. This is an analytical extraction for this paper; the identifiers refer to the standards documents and are not a replacement for them. Where §7.4 gives one row per obligation area, this table splits an area whose requirements would be discharged separately.

Area	Representative obligation	Appropriate discharge
Semantic transition	A complete closure is established and every supplied rule is evaluated.	Derivational plus observational evidence.
Construction	Construction refuses rather than repairs and produces no usable output on refusal.	Negative observational demonstration.
Construction	The same declarations and closure reproduce the same authorized identity.	Comparative demonstration.
Snapshot	Identity is derived from content; the snapshot is complete and immutable.	Derivational and structural demonstrations.
Snapshot acceptance	The four conditions of §4.1 are established before execution, and failure of any refuses the whole snapshot.	Negative observational and derivational demonstrations.
Execution	The runtime consumes only an accepted snapshot and does not add behavior from outside it.	Structural demonstration plus observational refusal fixtures.
Execution routing	Routing comes only from declared outcomes; an unrouted outcome refuses.	Negative observational demonstration.
Effects	External effects occur only through declared surfaces.	Structural absence demonstration.
Inspection	Inspection is read-only, does not execute, and answers about named artifacts.	Structural and observational demonstrations.
Evidence	Evidence identifies the snapshot, closure, rules, consequence, state, and path.	Derivational demonstration.

Area	Representative obligation	Appropriate discharge
Transformation	The baseline is named and frozen; claims about it are grounded through governed inspection.	Structural and observational demonstrations.
Transformation	Sufficiency precedes realization; a worker does not supply omitted meaning.	Negative observational demonstration.
Supersession	The predecessor becomes unreachable within the composition but remains retained and inspectable.	Structural and negative observational demonstrations.
Profile	The profile narrows the family and is not authored by the system claiming it.	Structural demonstration.

A complete conformance claim must state all applicable obligations and report uncovered obligations. Full demonstration coverage is not itself proof that demonstrations are adequate; demonstrations must also be capable of failure and use the correct discharge class.

Appendix C. Worked Lifecycle Example

Consider a governed platform whose baseline contains a declared workflow for approving a licensing action. A human-directed author proposes a change: the workflow must require an additional declared fact before approval and must record the resulting determination.

C.1 Transformation

The transformation begins with the named baseline snapshot. The author supplies the business decision and answers the questions that only the author can answer: what the additional fact means, which existing capability it constrains, what must remain preserved, and what outcome should occur when the fact is absent or contradictory.

The transformation grounds its claims about the existing workflow through governed inspection. It does not search the repository or infer that a similarly named artifact is the predecessor. The baseline identity and inspected artifact identities are recorded in the dossier. The new design is represented in registers. An unanswered blocking question makes the phase inadmissible rather than allowing a worker to fill it.

The derived design is checked for preservation and non-invention. A projection produces machine-readable candidate declarations only after its prior is admissible. Sufficiency is checked before realization: every field needed by the new capability contract, workflow routing, state transition, and evidence surface must be fixed by the design.

C.2 Construction and sealing

Construction admits the candidate declarations only if their references resolve, their governing rules hold, the workflow is structurally complete, the effect surfaces are declared, and the composition-wide obligations hold. If any candidate is inadmissible, construction refuses and produces no usable authorized representation.

The accepted representation is materialized into the selected projections. Verification checks that what is carried matches what was determined. Sealing then constitutes a snapshot with content-derived identity, complete constituent enumeration, provenance, integrity values, and a claim against an externally authored profile. The predecessor remains unchanged.

C.3 Acceptance and execution

The runtime accepts the successor snapshot only after verifying integrity, identity, totality, and profile. It receives an interaction in the canonical form of the governed boundary. The workflow traverses declared steps. The capability computes a declared outcome. The runtime reads declared routing for that outcome and applies the declared state transition and effect surface.

If the additional fact is absent, the interaction follows the declared refusal or negative outcome. If the capability produces an outcome not in its contract, the runtime does not

1
2
3
4 treat it as a route. It refuses and records the grounds. It does not retry or choose a
5 default.
6

7 The execution evidence identifies the successor snapshot, the applicable closure, the
8 rules and predicates, the resulting state, and the path. A later transformation may use
9 the evidence as an input to human judgment, but the evidence does not itself change
10 the system or authorize a new path.
11

12 C.4 What the example establishes 13

14 The example shows how the three activities compose without merging their authority. It
15 does not establish that the additional licensing fact is the right business decision, that
16 the capability implementation is correct, or that every possible effect path has been
17 structurally excluded. Those would require separate adequacy, implementation, and
18 negative-property demonstrations.
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65

Appendix D. Named-Revision Reproduction Record

This appendix records a clean reproduction of the composition identified in §8.1, performed at the composition's ordinal on the environment named below. The record is the run's output; reproducing the command text is not a claim that it was executed.

D.1 Subject identity

```
Standards revision:      v0
Public composition:      v4
Composition DOI:         10.5281/zenodo.22714911
Composition ordinal:     16      (see §8.1 – build at this ordinal, not the
                                workspace counter, or the identity differ
                                s)
Install distribution:     4.0.0
Profile in force:         GOVERNANCE_SURFACE_PROFILE_V0
Domains sealed:          7
Constituents:            595
Artifacts:               410
Snapshot identity:        d92b447fd39eaf926bb2f4330f9efbb19d744ecfbc3fe9a214a
                                e52945918905d
                                (snapshot_id and composite_hash identical)
Identity covers:          domains, constituents, profile
                                (provenance excluded – see §7.2 on profile content)
Component commits:        recorded in the deposit's MANIFEST.md, one row per
                                component beside its version DOI
Execution environment:    macOS 26.6.2, arm64; CPython 3.12.10 in an isolated
                                virtual environment; click, pyyaml, jsonschema only
Procedure:                regression.sh --all, 19 check steps, exit 0
```

The snapshot identity above is the identity the deposit carries, and the deposit is the reference against which a reproduction is judged: a run that does not reproduce it has not reproduced this composition. This record reproduces it. Nothing here is carried forward from an earlier run; a figure so carried would be a provenance record, not evidence about this one.

Reproducing at the composition's ordinal. The ordinal is carried in a compiled constituent, so a build at the workspace counter rather than at the composition's seals a different identity over byte-identical governed content (§8.1). At the released identity, the ordinal is the value each component repository declares at its v4 tag. Restoring that declaration — rather than editing it — is what makes the reproduction a reproduction.

D.2 The procedure

All paths are relative to the workspace root. One command performs the clean rebuild, every check, and the execution block:

```
cd <workspace-root>
bash .github/process/regression.sh --all
```

D.3 What the procedure expands to

Recorded so that a reader can see what is being claimed, not as an alternative to running D.2. The compile and assemble steps below are preceded by the cleanup that makes the build clean — deletion of every generated snapshot including each domain's, removal of data/ and traces/, and a workspace-wide .DS_Store sweep. Running the compile sequence without that cleanup does not reproduce this record.

```
export PGC_SNAPSHOT_PROFILE=GOVERNANCE_SURFACE_PROFILE_V0

./protocol_compiler/compile.sh STRUCTURE_BUILD_PLATFORM_CONFIG_V1
./protocol_compiler/compile_domain.sh conformance_workloads/workloads/collatz
./protocol_compiler/compile_domain.sh transformation
./protocol_compiler/compile_domain.sh snapshot_inspector
./protocol_compiler/compile_domain.sh business_domains/ai_governance
./protocol_compiler/compile_domain.sh business_domains/book_library_mgmt
./protocol_compiler/compile_domain.sh business_domains/blockchain
./snapshot_assembler/assemble.sh
```

The final paper must archive the stdout, stderr, assembled snapshot identity, execution traces, refusal outputs, and the enumerated red-by-design findings. It must not summarize --all as clean.

D.4 Findings to enumerate

A reproduction record states its red results rather than a pass rate. Three categories must be enumerated from the run, not carried over from an earlier one:

1. **Red by design.** Checks the runbook designates as expected-red, with the count and the reason each is deliberate. admission_contract_fidelity is the principal case. A run in which none of these is red indicates that a check has stopped reporting, and is a defect in the procedure rather than an improvement in the subject.
2. **Red and diagnosed.** Failures with a known cause, reported with that cause. These belong in §9.3 as threats, not in §8.5 as demonstrations.
3. **Red and undiagnosed.** Failures whose cause has not been established. These must be listed individually. An undiagnosed failure is the one category that cannot be summarized, because the summary would assert something about a cause nobody has determined.

Recorded results. Captured from the run this appendix documents; every figure reported in §8 traces to a line here.

snapshot identity	d92b447f...905d, matching the deposit assembler ordinal 16, profile GOVERNANCE_SURFACE_PROFILE_V0, 7 domains, 595 constituents
test_warm_boot.py	6/6 passed
test_governance_provenance.py	4/4 passed (DETERMINISM, ISOLATION, SENSITIVITY, ENFORCEMENT); provenance restored

```

1
2
3
4 si snapshot validate      valid: true, 10 checks, 0 non-advisory viola
5 tions
6
7      advisory red: republished_copies_agree
8
9      15
10      advisory red: bound_paths_declared_as_stores
11      1
12 admission_contract_fidelity 31 findings over 402 authored artifacts
13                             compared (expected-red; see §8.3)
14 construction_acceptance    99/99 artifacts reproduced across 4 domains,
15                             0 field differences
16 regression.sh --all        exit 0

```

The two advisory reds and the 31 expected-red findings are reported, not resolved. A run of this composition that reported none of them would indicate that a check had stopped reporting, which is a defect in the procedure rather than an improvement in the subject (§8.3).

Where a runbook's prose and a run disagree, the run is the evidence. Two rows above were, at an earlier revision, carried in the runbook's expected-results table with values that no longer reproduced — `test_warm_boot.py` recorded as two failures and `test_governance_provenance.py` as two of four red. Both now pass, and the discrepancy was found by running rather than by reading.

Appendix E. Claim Provenance Matrix

Claim in this paper	Primary source	Status	Evidence boundary
Governance, construction, and execution are three activities over one transition schema.	PGC Conceptual Model; PGC Semantic Model; the realization preprint.	Prior plus synthesis.	Architectural derivation; not an implementation claim.
Execution realizes sealed declared structure and refuses when declarations run out.	PGC Execution Model; deterministic declarative execution preprint.	Prior.	Requires execution and negative demonstrations for a realization.
Construction determines admissibility, not adequacy, and refuses rather than repairs.	PGC Governed Construction.	Normative specification.	Requires construction refusal and no-output demonstrations.
Transformation changes a named baseline through governed phases and human-directed meaning.	PGC Governed Transformation; closed-loop transformation preprint.	Prior plus synthesis.	Requires grounded baseline and transformation evidence.
A snapshot is sealed, complete, content-identified, self-describing, and verifiable.	PGC Snapshot.	Normative specification.	Requires derivational and structural evidence.
A profile is external to the system claiming it and narrows the family.	PGC Normative Platform Profile.	Normative specification.	Requires authorship and profile-constraint evidence.
The reference realization can build and exercise the named composition.	The v4 composition deposit (10.5281/zenodo.22714911) and its runbook.	Implementation observation.	Limited to the deposited composition, its profile, environment, and workloads.
The reference realization fully conforms.	None.	Not established.	Red-by-design checks, single-implementation acceptance, and absence of

Claim in this paper	Primary source	Status	Evidence boundary
PGC improves business correctness, security, or maintenance economics.	None in this paper.	Not established.	comparative evidence preclude this claim. Requires separate adequacy, threat, or longitudinal studies.

Declaration of generative AI use

The author is the sole author of this manuscript. Its research content — the architectural argument, the lifecycle synthesis, the conformance framing, the claims made and the limits placed on them — was conceived and written by the author, who takes full responsibility for it.

A generative AI assistant (Claude Opus 5, Anthropic) was used in three supporting capacities, none of which originated the paper’s content or claims:

1. **As a coding agent on the reference implementation.** The assistant performed construction work under the governed process this paper describes, in the role the architecture assigns to a worker: it derived and realized artifacts from designs the author authored and admitted, and it held no authority to establish scope, admit a candidate, or promote a baseline. No claim is made here about the assistant’s capability, productivity, or output quality; the architecture is worker-neutral by design, and the relevant property is that changing the worker does not change who holds authority.
2. **Extracting evidence from the reference implementation and integrating it with the text.** Figures reported in §8 and Appendix D — identities, counts, check results — were read from build and run outputs and transcribed into the manuscript. Every such figure is reproducible from the deposited composition by the procedure in Appendix D, and the run output, not the manuscript, is the evidence.
3. **Proofreading and formatting.**

No text was generated for inclusion without the author’s review, revision, and acceptance. The assistant is not credited as an author, consistent with the position that authorship entails accountability that cannot be held by a tool.

References

Primary PGC preprints

Ganti, B. *Protocol-Governed Computing: An Architecture for Deterministic Declarative Execution*. 2026. DOI: [10.5281/zenodo.21879516](https://doi.org/10.5281/zenodo.21879516).

Ganti, B. *Protocol-Governed Computing: An Architecture for Closed-Loop Governed Transformation*. 2026. DOI: [10.5281/zenodo.21879948](https://doi.org/10.5281/zenodo.21879948).

Ganti, B. *Protocol-Governed Computing: Realizing the Normative Platform and Its Governed Transformation*. 2026. DOI: [10.5281/zenodo.21880155](https://doi.org/10.5281/zenodo.21880155).

Standards and realization

Protocol-Governed Computing. *PGC Standards Family*, revision v0. In particular: Conceptual Model; Semantic Model; Architectural Invariants; Governed Construction; Execution Model; Snapshot; Evidence, Attestation & Provenance; Governed Transformation; Supersession; Normative Platform Profile; Conformance Model; Conformance Test Specification. 2026. DOI: [10.5281/zenodo.22150616](https://doi.org/10.5281/zenodo.22150616).

Protocol-Governed Computing. *Reference Implementation: the composed platform, v4*. Sealed snapshot d92b447f...905d, assembler ordinal 16, profile GOVERNANCE_SURFACE_PROFILE_V0. 2026. DOI: [10.5281/zenodo.22714911](https://doi.org/10.5281/zenodo.22714911). The deposit names its nine component archives as parts, each with its own version DOI. This is the artifact the assessment in §8 concerns.

Protocol-Governed Computing. *End-to-End Runbook*. Repository revision 5ad8a62. <https://github.com/protocol-governed-computing>.

Protocol-Governed Computing. *pgc_install*. Current repository revision, accessed 2026-09-11. <https://github.com/protocol-governed-computing>.

Earlier PGS papers

Ganti, B. *Protocol-Governed Systems: Architecture Inversion Concepts*. 2026. DOI: [10.5281/zenodo.20497732](https://doi.org/10.5281/zenodo.20497732).

Ganti, B. *Protocol-Governed Systems: Compiler Conceptual Model*. 2026. DOI: [10.5281/zenodo.20471804](https://doi.org/10.5281/zenodo.20471804).

Ganti, B. *Protocol-Governed Systems: A Conceptual Model*. 2026. DOI: [10.5281/zenodo.20300611](https://doi.org/10.5281/zenodo.20300611).

Ganti, B. *Protocol-Governed Systems: A Constitutionally Constrained Architecture for Autonomous and AI-Generated Software*. 2026. DOI: [10.5281/zenodo.2033650](https://doi.org/10.5281/zenodo.2033650).

Ganti, B. *Protocol-Governed Systems: Runtime Conceptual Model*. 2026. DOI: [10.5281/zenodo.20478471](https://doi.org/10.5281/zenodo.20478471).

Ganti, B. *Protocol-Governed Computing: Field Manual*. 2026. DOI: [10.5281/zenodo.21898082](https://doi.org/10.5281/zenodo.21898082).

Concurrent submission by the same author

Ganti, B. *Protocol-Governed Human-AI Software Engineering: Autonomy Without Authority*. 2026. DOI: [10.5281/zenodo.22650863](https://doi.org/10.5281/zenodo.22650863). Under concurrent review elsewhere. Cited for disclosure and positioning; no text, argument, data, or evidence from it is reused in this paper. See §1.3.

Related work

Brambilla, M., Cabot, J., and Wimmer, M. (2017). *Model-Driven Software Engineering in Practice*, 2nd ed. Synthesis Lectures on Software Engineering. Morgan & Claypool.

Bucchiarone, A., Cabot, J., Paige, R. F., and Pierantonio, A. (2020). Grand challenges in model-driven engineering: an analysis of the state of the research. *Software and Systems Modeling*, 19(1), 5–13. DOI: [10.1007/s10270-019-00773-6](https://doi.org/10.1007/s10270-019-00773-6).

Dennis, J. B., and Van Horn, E. C. (1966). Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3), 143–155.

Haas, A., Rossberg, A., Schuff, D. L., Titzer, B. L., Holman, M., Gohman, D., Wagner, L., Zakai, A., and Bastien, J. F. (2017). Bringing the web up to speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017)*, 185–200.

Jackson, D. (2006). *Software Abstractions: Logic, Language, and Analysis*. MIT Press. ISBN 978-0-262-10114-1.

Lamb, C., and Zacchiroli, S. (2022). Reproducible builds: increasing the integrity of software supply chains. *IEEE Software*, 39(2), 62–70.

Meyer, B. (1992). Applying “design by contract.” *Computer*, 25(10), 40–51. DOI: [10.1109/2.161279](https://doi.org/10.1109/2.161279).

Miller, M. S. (2006). *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University.

Newman, Z., Meyers, J. S., and Torres-Arias, S. (2022). Sigstore: software signing for everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)*, 2353–2367.

Open Policy Agent (2016–). *Open Policy Agent: policy-based control for cloud-native environments*. Cloud Native Computing Foundation project.

OpenSSF (2023). *Supply-chain Levels for Software Artifacts (SLSA)*, version 1.0, released 19 April 2023. Open Source Security Foundation.

Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12), 1053–1058. DOI: [10.1145/361598.361623](https://doi.org/10.1145/361598.361623).

Rahman, A., Parnin, C., and Williams, L. (2019). The seven sins: security smells in infrastructure as code scripts. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, 164–175.

Rahman, A., Farhana, E., Parnin, C., and Williams, L. (2020). Gang of eight: a defect taxonomy for infrastructure as code scripts. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE 2020)*, 752–764.

Torres-Arias, S., Afzali, H., Kuppusamy, T. K., Curtmola, R., and Cappos, J. (2019). in-toto: providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium*, 1393–1410.

Declaration of interests

☐The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

☒The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Bhash Ganti reports was provided by None. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.
